

Neural Network Training

Neural Network Training

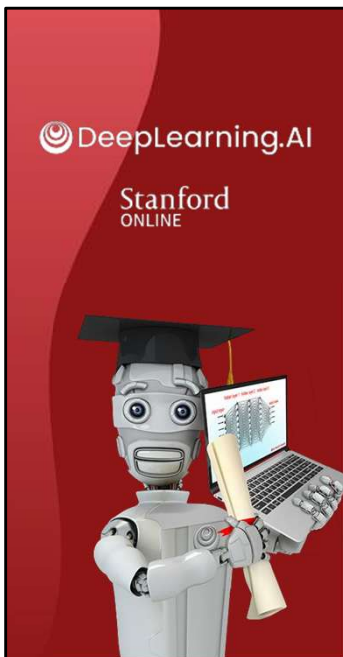
This chapter focuses on how a neural network learns from data. Instead of just computing predictions (forward propagation), we now allow the model to adjust its weights and biases so that its predictions become more accurate. Training means minimizing the error between predicted outputs and the true labels using an optimization algorithm such as gradient descent.

TensorFlow implementation

In this section, we move from theory to practice. We use TensorFlow to define the model, choose a loss function, select an optimizer, and train the network using `model.fit`. TensorFlow automatically computes gradients and updates the parameters, making the training process efficient and scalable.

Content

- Neural Network Training
- Activation Functions
- Multiclass Classification
- Additional Neural Network Concepts
- Back Propagation (Optional)



Week 2

Neural Network Training

TensorFlow implementation

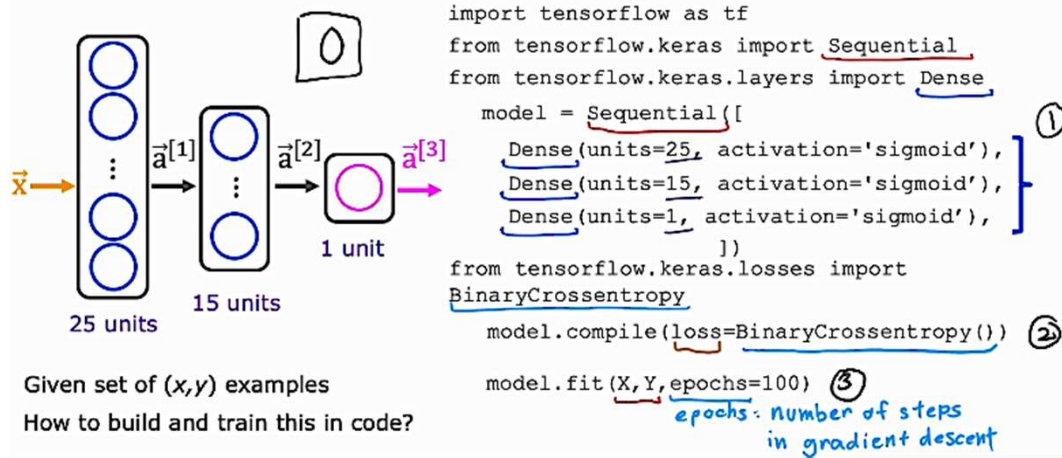
Neural Network Training

This chapter focuses on how a neural network learns from data. Instead of just computing predictions (forward propagation), we now allow the model to adjust its weights and biases so that its predictions become more accurate. Training means minimizing the error between predicted outputs and the true labels using an optimization algorithm such as gradient descent.

TensorFlow implementation

In this section, we move from theory to practice. We use TensorFlow to define the model, choose a loss function, select an optimizer, and train the network using `model.fit`. TensorFlow automatically computes gradients and updates the parameters, making the training process efficient and scalable.

Train a Neural Network in TensorFlow



Train a Neural Network in TensorFlow

This slide shows how we move from defining a neural network to actually training it using TensorFlow.

First, we import TensorFlow and the needed tools such as Sequential and Dense. Sequential means the layers are connected in order, one after another. Dense means a fully connected layer where every neuron receives input from all neurons in the previous layer.

The model is defined as `Sequential([ Dense(units=25, activation="sigmoid"), Dense(units=15, activation="sigmoid"), Dense(units=1, activation="sigmoid") ])`.

This creates a neural network with two hidden layers (25 units and 15 units) and one output layer (1 unit). The sigmoid activation is used to produce values between 0 and 1, which can represent probabilities.

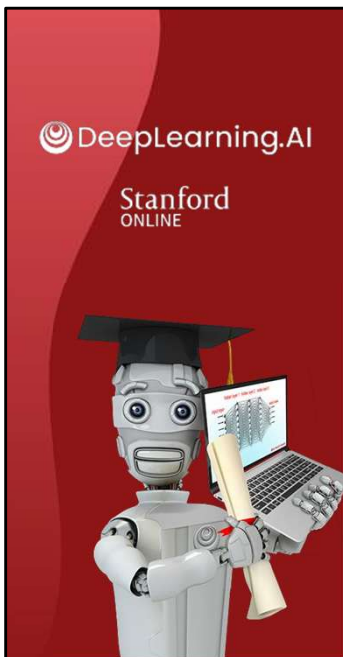
Next, we import `BinaryCrossentropy`. This is the loss function used for binary classification problems. It measures how far the predicted output is from the true label.

Then we prepare the model using `model.compile(loss=BinaryCrossentropy())`. This step tells TensorFlow how to measure error during training.

Finally, we train the model using `model.fit(X, Y, epochs=100)`.

Here, X is the input data, Y is the correct labels, and `epochs=100` means the training process will run 100 times over the entire dataset. In each epoch, gradient descent adjusts the weights to reduce the loss.

The key idea of this slide is that training a neural network in TensorFlow follows three main steps: define the model, compile it with a loss function, and train it using `model.fit()`.



Neural Network Training

Training Details

Training Details

In this section, we move from building a neural network to understanding how it actually learns. Training details explain what happens inside the learning process: how the model measures its mistakes, how it adjusts its weights, and how this process is repeated many times to improve performance.

Here we focus on important concepts such as the loss function, gradient descent, epochs, and how TensorFlow updates the parameters step by step. The goal of this section is to clearly understand what training really means and what is happening behind `model.fit(x, y)`.

Model Training Steps TensorFlow

① specify how to compute output given input x and parameters w, b (define model) $f_{\vec{w}, b}(\vec{x}) = ?$	logistic regression <pre>z = np.dot(w, x) + b f_x = 1 / (1 + np.exp(-z))</pre>	neural network <pre>model = Sequential([Dense(...), Dense(...), Dense(...)])</pre>
② specify loss and cost $L(f_{\vec{w}, b}(\vec{x}), y)$ 1 example $J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)})$	logistic loss <pre>loss = -y * np.log(f_x) - (1-y) * np.log(1-f_x)</pre>	binary cross entropy <pre>model.compile(loss=BinaryCrossentropy())</pre>
③ Train on data to minimize $J(\vec{w}, b)$	<pre>w = w - alpha * dj_dw b = b - alpha * dj_db</pre>	<pre>model.fit(X, y, epochs=100)</pre>

DeepLearning.AI Stanford ONLINE

Model Training Steps

This slide summarizes the three main steps needed to train any machine learning model, whether it is logistic regression or a neural network in TensorFlow.

Step 1: Define the model.

First, we specify how the model computes its output from the input x and the parameters (weights and biases). In logistic regression, we compute $z = w \cdot x + b$, then apply the sigmoid function to get the prediction f_x . In a neural network, we define layers using `Sequential` and `Dense`, which automatically perform these computations layer by layer.

Step 2: Define the loss (cost) function.

Next, we measure how good or bad the predictions are. For binary classification, logistic regression uses logistic loss. In neural networks, we use `BinaryCrossentropy`. The loss tells us how far the predicted value is from the true label y . The cost function is the average loss over all training examples.

Step 3: Train the model (minimize the cost).

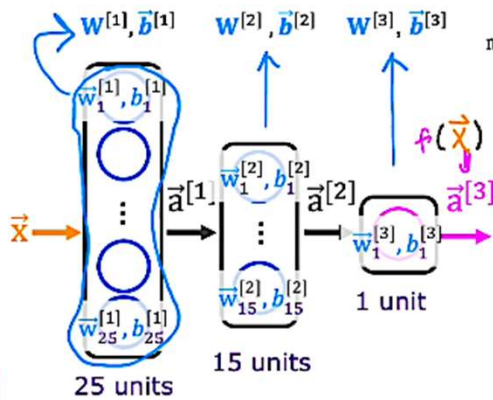
Finally, we adjust the weights and biases to reduce the cost. In logistic regression, this is done using gradient descent with update rules like $w = w - \alpha * \text{derivative}$. In TensorFlow, this process is handled automatically when we call `model.fit(X, Y, epochs=...)`. TensorFlow computes gradients and updates parameters internally.

The key idea is that training always follows the same structure: define the model, define the loss, then optimize the parameters to minimize that loss.

1. Create the model

define the model

$$f(\vec{x}) = ?$$



```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='sigmoid'),
    Dense(units=15, activation='sigmoid'),
    Dense(units=1, activation='sigmoid'),
])
```

Create the Model

This slide explains the first step in training a neural network: defining the model $f(x)$. In simple words, we must tell the computer what the structure of the neural network looks like before we can train it.

On the left side, the diagram shows the architecture. The input x goes into the first hidden layer with 25 units. Each unit has its own weights and bias, written as $w[1]$ and $b[1]$. The output of this layer becomes $a[1]$. Then $a[1]$ goes into the second hidden layer with 15 units, which has its own parameters $w[2]$ and $b[2]$, producing $a[2]$. Finally, the output layer has 1 unit with parameters $w[3]$ and $b[3]$, and it produces $a[3]$, which is the final prediction $f(x)$.

On the right side, we implement this structure in TensorFlow. We first import TensorFlow and the needed classes. Then we define the model using:

```
model = Sequential([
    Dense(units=25, activation="sigmoid"),
    Dense(units=15, activation="sigmoid"),
    Dense(units=1, activation="sigmoid")
])
```

This code builds the same architecture shown in the diagram. Each Dense layer automatically creates its own weights and biases. The data will flow through the layers in order, from input to output.

The key idea of this slide is that creating the model means defining the function $f(x)$: specifying how the input moves through layers and how predictions are computed.

2. Loss and cost functions

handwritten digit classification problem → binary classification

$$L(f(\vec{x}), y) = -y \log(f(\vec{x})) - (1 - y) \log(1 - f(\vec{x}))$$

Compare prediction vs. target

logistic loss
also known as binary cross entropy

$$J(W, B) = \frac{1}{m} \sum_{i=1}^m L(f(\vec{x}^{(i)}), y^{(i)})$$

$W^{[1]}, W^{[2]}, W^{[3]}$ $\vec{b}^{[1]}, \vec{b}^{[2]}, \vec{b}^{[3]}$ $f_{W,B}(\vec{x})$

```

model.compile(loss= BinaryCrossentropy())
from tensorflow.keras.losses import BinaryCrossentropy

```

regression (predicting numbers and not categories) mean squared error

```

model.compile(loss= MeanSquaredError())
from tensorflow.keras.losses import MeanSquaredError

```

K Keras

This slide explains how we measure the performance of a neural network during training.

In a binary classification problem, such as handwritten digit classification (for example, predicting 0 or 1), the model outputs a prediction $f(x)$. We then compare this prediction with the true label y . The loss function for one example is $L(f(x), y) = -y \log(f(x)) - (1 - y) \log(1 - f(x))$. This is called logistic loss, also known as binary cross entropy. It measures how far the prediction is from the correct answer.

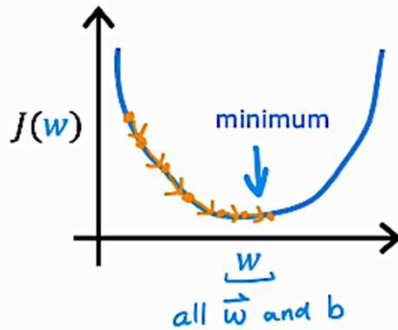
To evaluate the whole training set, we compute the cost function $J(W, B)$, which is the average of the loss over all m training examples. So the cost tells us, on average, how well the model is performing.

In TensorFlow, we specify this using `model.compile(loss=BinaryCrossentropy())`. This tells the model to use binary cross entropy during training.

For regression problems, where we predict numbers instead of categories, we use a different loss called mean squared error, and in TensorFlow we write `model.compile(loss=MeanSquaredError())`.

The key idea is that the loss function measures error for one example, and the cost function is the average error over the entire dataset.

3. Gradient descent



repeat {

$$w_j^{[l]} = w_j^{[l]} - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b_j^{[l]} = b_j^{[l]} - \alpha \frac{\partial}{\partial b_j} J(\vec{w}, b)$$

} Compute derivatives
for gradient descent
using "backpropagation"

`model.fit(X, y, epochs=100)`

DeepLearning.AI Stanford ONLINE

Gradient Descent

This slide explains how the neural network actually learns. After we define the model and the loss function, we need a way to reduce the cost $J(W, b)$. Gradient descent is the algorithm used to minimize this cost.

On the left, the graph shows the cost $J(W)$ on the vertical axis and the parameters (all weights and biases) on the horizontal axis. The curve has a minimum point. Gradient descent starts from an initial value of W and b and takes small steps downhill until it reaches a minimum.

On the right, the update rules are shown. For every layer l , each weight $w[l]$ and bias $b[l]$ is updated as

$$w[l] = w[l] - \alpha * \partial J / \partial w[l]$$

$$b[l] = b[l] - \alpha * \partial J / \partial b[l]$$

Here, α is the learning rate. It controls how big each step is. The partial derivatives tell us the direction in which the cost increases. We subtract them to move in the direction that decreases the cost.

The derivatives are computed using backpropagation. In TensorFlow, this whole process happens automatically when we call `model.fit(X, Y, epochs=100)`. The model repeatedly updates its weights over many epochs until the cost becomes small.

Key idea: Gradient descent adjusts all weights and biases step by step to minimize the cost function and improve predictions.

Neural network libraries

Use code libraries instead of coding "from scratch"

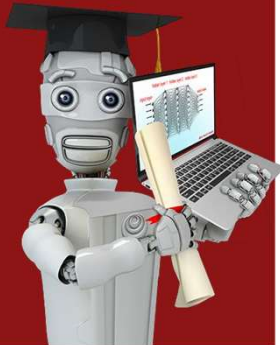


Good to understand the implementation
(for tuning and debugging).

This slide explains that in real-world machine learning, we usually do not build neural networks completely from scratch. Instead, we use powerful libraries such as TensorFlow and PyTorch. These libraries already provide ready-made tools for building layers, defining loss functions, computing gradients, and performing optimization.

Using these libraries saves time and reduces errors, especially for large and complex models. However, it is still very important to understand how neural networks work internally. Knowing the underlying mathematics and training process helps us tune hyperparameters, debug problems, and improve model performance.

The key idea is that libraries make implementation easier, but strong conceptual understanding makes you a better machine learning engineer.



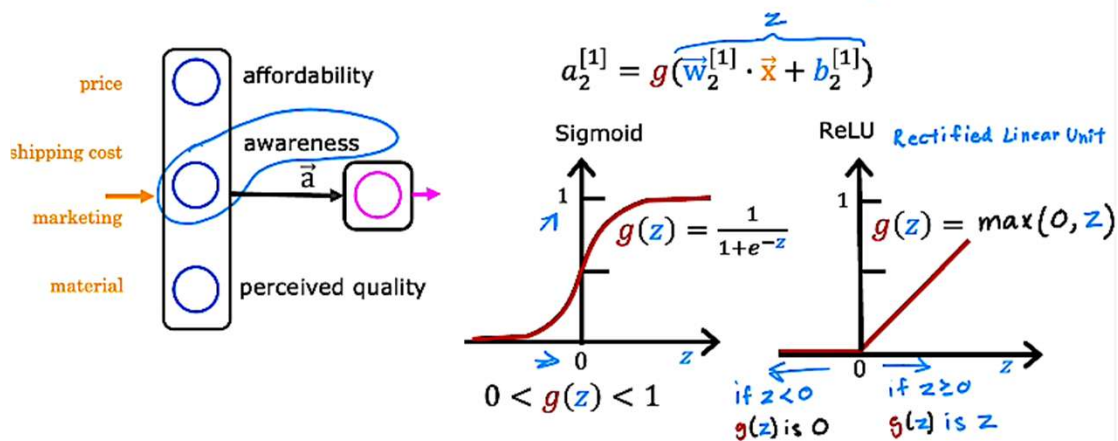
Activation Functions

Alternatives to the sigmoid activation

Activation Functions is a new section that focuses on how neurons decide their output values. Up to now, we mainly used the sigmoid function, but it is not the only choice.

Alternatives to the sigmoid activation means we will study other activation functions that can replace sigmoid in hidden layers. Different activation functions affect how the network learns, how fast it trains, and how well it performs. In this section, we explore why sigmoid is sometimes not ideal and introduce better options such as ReLU and others that improve training in deeper neural networks.

Demand Prediction Example



DeepLearning.AI Stanford ONLINE

Activation Functions – Alternatives to Sigmoid (Demand Prediction Example)

This slide compares two activation functions: Sigmoid and ReLU, using the demand prediction example.

In the neural network, each neuron first computes a linear value z using the equation $z = w \cdot x + b$. Then we apply an activation function $g(z)$ to produce the final output a . The activation function introduces non-linearity, which allows the network to learn complex patterns.

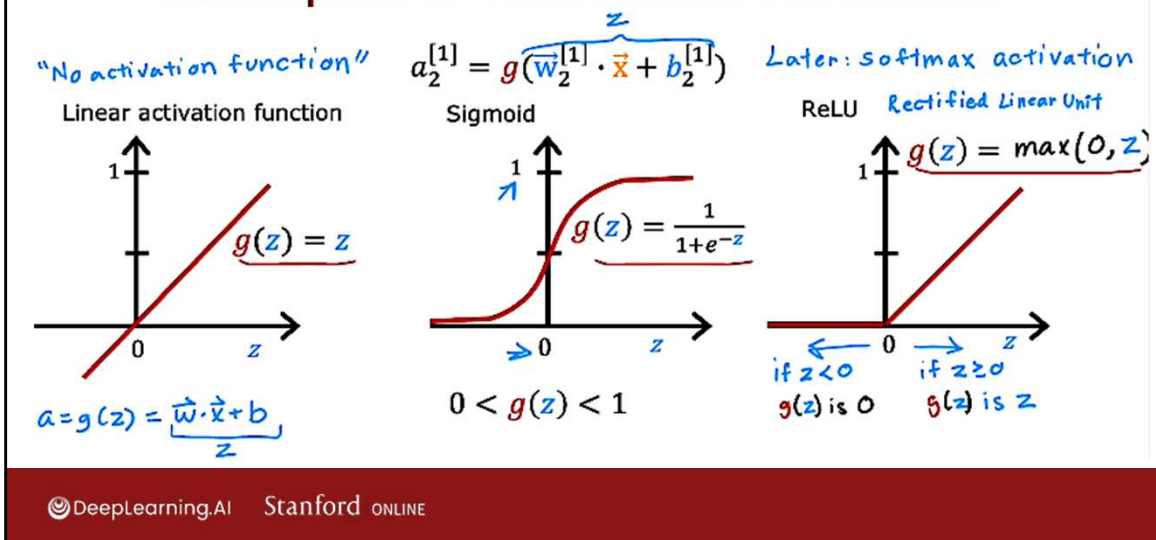
The Sigmoid function is defined as $g(z) = 1 / (1 + e^{(-z)})$. Its output is always between 0 and 1. This makes it very useful in the output layer for binary classification problems, such as predicting whether a product will be a top seller or not. However, sigmoid can become very flat for large positive or negative values of z , which may slow down learning.

ReLU (Rectified Linear Unit) is defined as $g(z) = \max(0, z)$. If z is negative, the output is 0. If z is positive, the output equals z . ReLU is simple and computationally efficient. It helps training deeper networks because it reduces the vanishing gradient problem and allows faster learning.

In practice, sigmoid is often used in the output layer for binary classification, while ReLU is commonly used in hidden layers to improve training performance.

Key idea: Activation functions transform the linear combination z into a meaningful output, and choosing the right activation function affects how well and how fast the network learns.

Examples of Activation Functions



Examples of Activation Functions

This slide compares three common activation functions used in neural networks: linear, sigmoid, and ReLU. Each activation function takes the value z , which is computed as $w \cdot x + b$, and transforms it into an output $a = g(z)$.

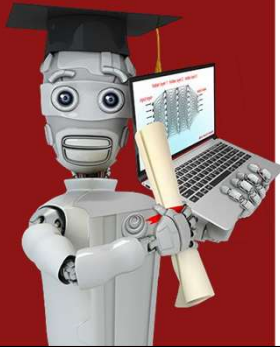
The linear activation function simply returns $g(z) = z$. This means the output is exactly the same as the input. If we use only linear activations in all layers, the entire network behaves like a linear model, even if it has many layers.

Because of this, linear activation is usually not used in hidden layers.

The sigmoid activation function is defined as $g(z) = 1 / (1 + e^{(-z)})$. Its output is always between 0 and 1. This makes it very useful for binary classification problems, especially in the output layer, where we want to interpret the result as a probability.

The ReLU activation function is defined as $g(z) = \max(0, z)$. If z is negative, the output is 0. If z is positive, the output equals z . ReLU is widely used in hidden layers because it is simple, efficient, and helps deep networks train faster.

The key idea is that the activation function introduces non-linearity into the network. Without it, the neural network cannot learn complex patterns.



Activation Functions

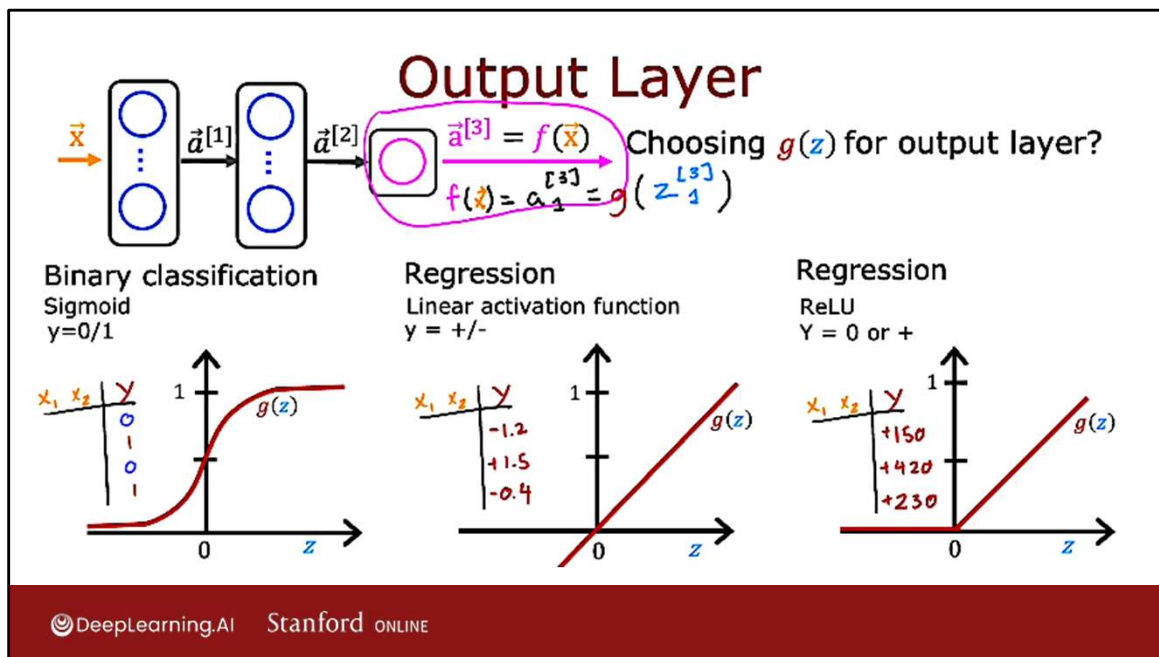
Choosing activation function

Choosing activation functions is an important design decision when building a neural network. Different activation functions change how neurons behave and what kind of patterns the network can learn.

In hidden layers, ReLU is usually a good default choice because it is simple, fast to compute, and works well for deep networks. It helps reduce the vanishing gradient problem and allows the network to learn complex patterns more easily.

Sigmoid is often used in the output layer for binary classification problems because its output is between 0 and 1, which can be interpreted as a probability. However, sigmoid is usually not preferred in deep hidden layers because it can slow down learning.

For regression problems (predicting numbers), a linear activation is commonly used in the output layer. For multi-class classification, softmax is typically used in the output layer to produce probabilities across multiple classes. The key idea is: use ReLU in hidden layers in most cases, choose sigmoid or softmax in the output layer for classification, and use linear activation for regression.



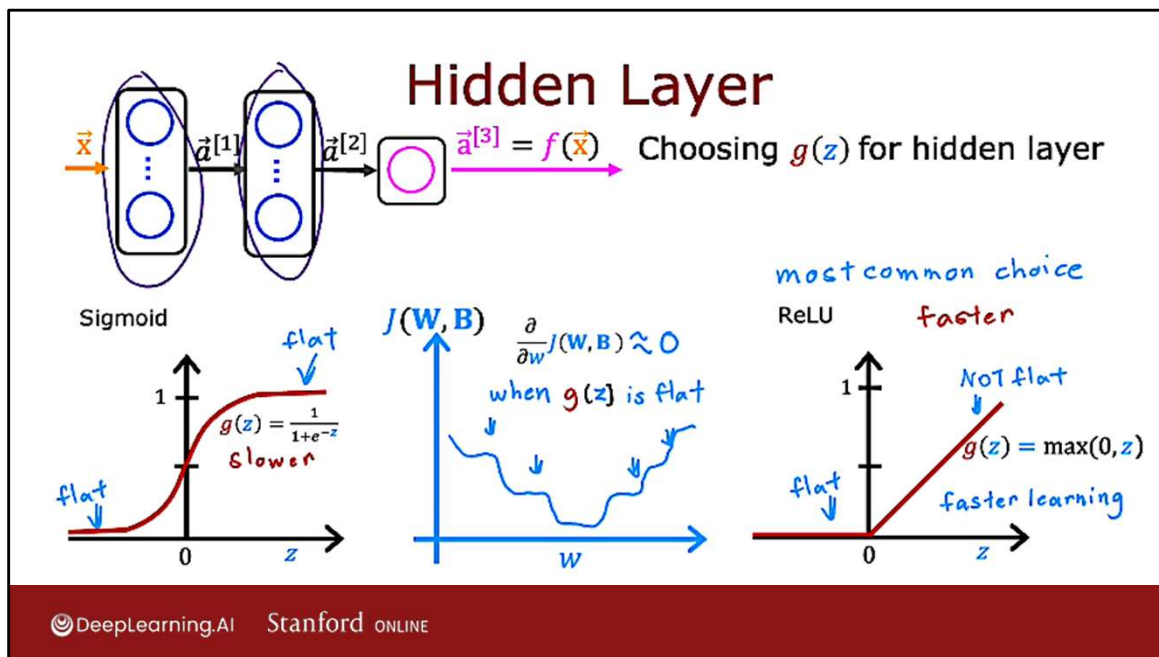
Output Layer (Choosing the Activation Function)

This slide explains how to choose the activation function for the output layer based on the type of problem. The output layer produces the final prediction of the network, written as $a[L] = g(z[L])$. The function $g(z)$ depends on whether we are solving classification or regression.

For binary classification (for example $y = 0$ or 1), we use the sigmoid activation function. Sigmoid outputs values between 0 and 1 , so it can be interpreted as a probability. If the output is close to 1 , the model predicts class 1 ; if it is close to 0 , it predicts class 0 .

For regression problems where the target can be any real number (positive or negative), we usually use a linear activation function, meaning $g(z) = z$. This allows the network to produce unrestricted values, such as -1.2 , 3.5 , or 10.7 . In some regression cases where the target must be non-negative (for example prices or counts), ReLU can be used in the output layer. ReLU outputs 0 when z is negative and z when z is positive, so predictions are always greater than or equal to zero.

The key idea is that the output layer activation must match the type of prediction we want: sigmoid for binary classification, linear for general regression, and sometimes ReLU when outputs must be non-negative.



Hidden Layer – Choosing the Activation Function

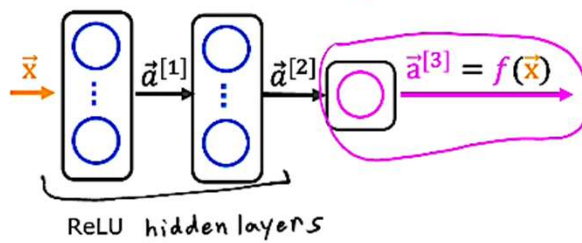
This slide explains how to choose the activation function for hidden layers in a neural network.

In the past, sigmoid was commonly used in hidden layers. However, sigmoid has a problem: when the input z is very large or very small, the function becomes flat. When it is flat, the gradient (derivative) becomes very small, meaning the weights update very slowly. This is called the vanishing gradient problem, and it makes training slower.

ReLU, defined as $g(z) = \max(0, z)$, is now the most common choice for hidden layers. ReLU does not saturate for positive values, so the gradient does not become very small in most cases. This allows faster learning and more efficient training.

Key idea: For hidden layers, ReLU is usually preferred because it trains faster and avoids the flat regions that slow down learning in sigmoid.

Choosing Activation Summary



```
from tf.keras.layers import Dense
model = Sequential([
    Dense(units=25, activation='relu'), layer1
    Dense(units=15, activation='relu'), layer2
    Dense(units=1, activation='sigmoid') layer3
])
```

or 'linear'

binary classification
 activation='sigmoid'
 regression y negative/
 activation='linear' positive
 regression $y \geq 0$
 activation='relu'

Choosing Activation Summary

This slide summarizes how to choose activation functions in a neural network. For hidden layers, the most common choice today is ReLU. ReLU is simple, fast, and helps the network learn faster because it does not become flat for positive values. In code, this means using `activation="relu"` in the Dense layers of the hidden part of the network.

For the output layer, the activation depends on the type of problem. In binary classification, where the output is 0 or 1, we usually use `activation="sigmoid"` because it gives a value between 0 and 1 that can be interpreted as a probability.

In regression problems, if the output can be any real number (positive or negative), we typically use `activation="linear"` (or no activation). If the output must be non-negative, we may use `activation="relu"` in the output layer. The key idea is simple: use ReLU for hidden layers in most cases, and choose the output activation based on the type of prediction you want to make.

Other Activation Functions

- Tanh (Hyperbolic Tangent)
 $g(z) = \tanh(z)$
Its output range is between -1 and 1. It is similar to sigmoid but centered at zero, which sometimes helps learning.
- Leaky ReLU
 $g(z) = \max(\alpha z, z)$ where α is a small number like 0.01.
Instead of outputting 0 for negative values (like ReLU), it allows a small negative slope. This helps reduce the “dying ReLU” problem.

Other Activation Functions

In addition to sigmoid and ReLU, there are several other activation functions that can be used in neural networks. Each one has different properties and may work better depending on the problem.

Tanh is similar to sigmoid but its output ranges from -1 to 1 instead of 0 to 1. Because it is centered around zero, it can sometimes help the model learn faster than sigmoid.

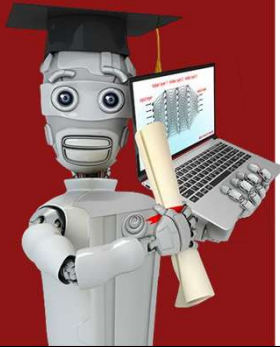
Leaky ReLU is a small modification of ReLU. Instead of giving 0 when z is negative, it gives a small negative value. This helps avoid neurons that completely stop learning.

ELU is another improvement over ReLU. It keeps positive values linear, but makes negative values smooth instead of flat. This can improve training stability.

Softmax is mainly used in the output layer for multi-class classification problems. It converts outputs into probabilities that add up to 1, which makes it easy to choose the predicted class.

Swish is a newer activation function defined as z times $\text{sigmoid}(z)$. It is smooth and sometimes performs better than ReLU in deep networks.

The key idea is that there is no single best activation function for all cases. The choice depends on the task, the network depth, and experimental results.



Activation Functions

Why do we need activation

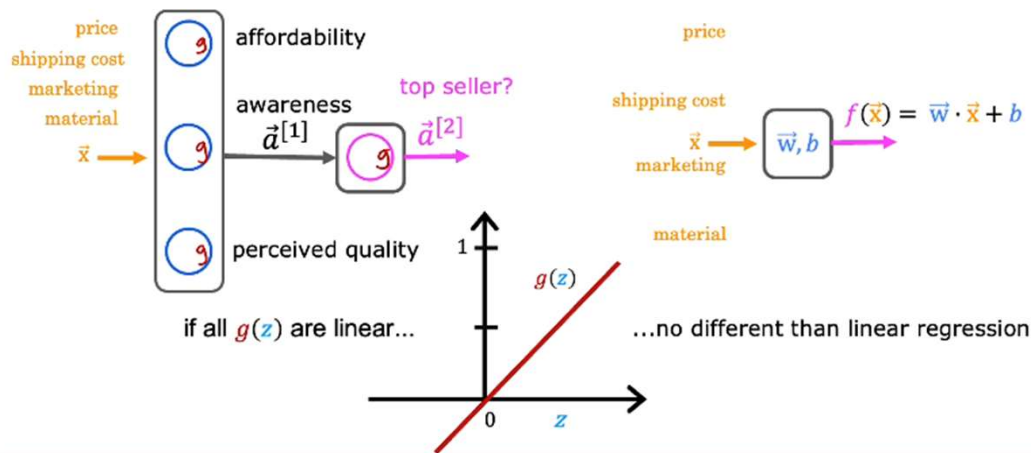
Activation functions are essential in neural networks because they introduce non-linearity into the model. Without an activation function, each layer would only perform a linear transformation of the input, meaning that even a deep network with many layers would behave like a single linear model. This would severely limit what the network can learn.

Real-world problems, such as image recognition, speech processing, and demand prediction, are not linear. The relationship between inputs and outputs is usually complex. Activation functions allow the network to learn these complex patterns by transforming the weighted sum $z = w \cdot x + b$ into a non-linear output $a = g(z)$.

Activation functions also help control the range of outputs. For example, sigmoid keeps outputs between 0 and 1, which is useful for probabilities in binary classification. ReLU sets negative values to zero and keeps positive values unchanged, which often helps training move faster and reduces some learning problems.

In short, activation functions make neural networks powerful. They allow deep models to learn complex relationships instead of behaving like simple linear regression.

Why do we need activation functions?



Why do we need activation functions?

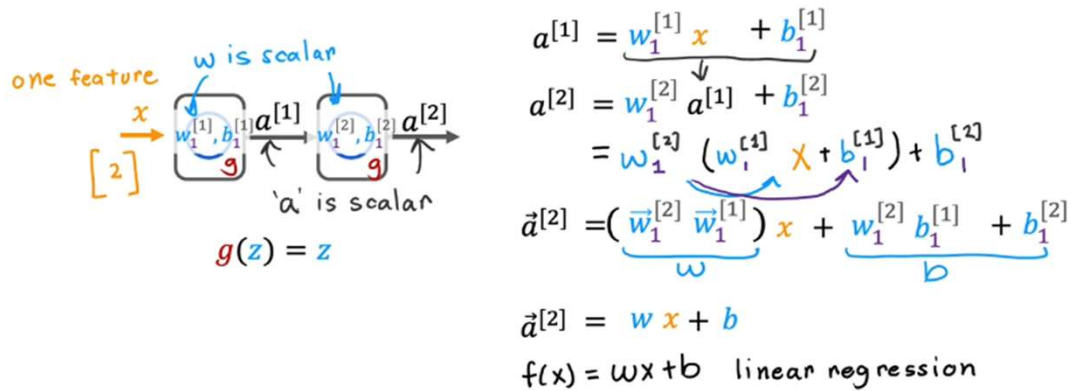
This slide explains a very important idea. If all activation functions in a neural network are linear, then the whole network becomes linear. Even if we stack many layers, the final result will still be just a linear function of the input, like $f(x) = w \cdot x + b$. That means the model is no more powerful than simple linear regression.

On the left side, we see several neurons computing features such as affordability, awareness, and perceived quality. If each neuron only applies a linear function $g(z) = z$, then combining many layers does not increase the model's power. The network cannot learn complex patterns such as non-linear decision boundaries.

Activation functions introduce nonlinearity into the model. This nonlinearity allows the network to learn complex relationships between inputs and outputs. For example, it can learn curved boundaries, interactions between features, and hidden patterns in the data.

Key idea: Without activation functions, deep neural networks collapse into a single linear model. Activation functions are what make neural networks truly powerful and capable of learning complex real-world problems.

Linear Example



DeepLearning.AI Stanford ONLINE

Linear Example

This slide shows what happens if we use only linear activation functions, where $g(z) = z$, in all layers of a neural network.

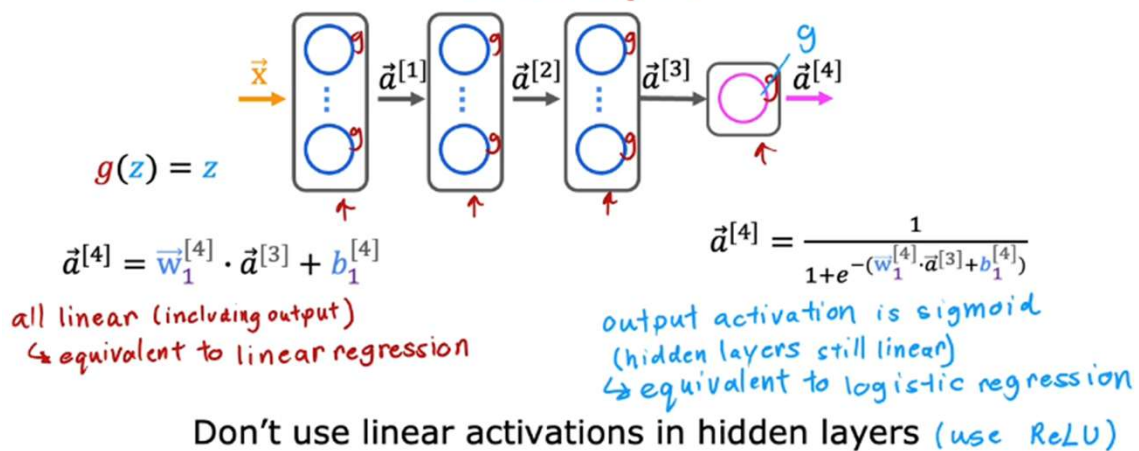
We start with one input feature x , so x is just a number (a scalar). In the first layer, we compute $a[1] = w[1]x + b[1]$. Since x is a scalar and $w[1]$ is also a scalar, the output $a[1]$ is still a scalar.

In the second layer, we compute $a[2] = w[2]a[1] + b[2]$. If we substitute $a[1]$ into this equation, we get $a[2] = w[2](w[1]x + b[1]) + b[2]$. When we expand this, it becomes $a[2] = (w[2]w[1])x + (w[2]b[1] + b[2])$.

So even though we used two layers, the final result is still of the form $w x + b$. This is exactly a linear function, the same as linear regression.

The key idea is that stacking multiple layers with linear activation functions does not make the model more powerful. It is still just one linear model. That is why we need non-linear activation functions in hidden layers, so the network can learn more complex patterns.

Example



This slide gives a clear example of why we should not use linear activation functions in hidden layers.

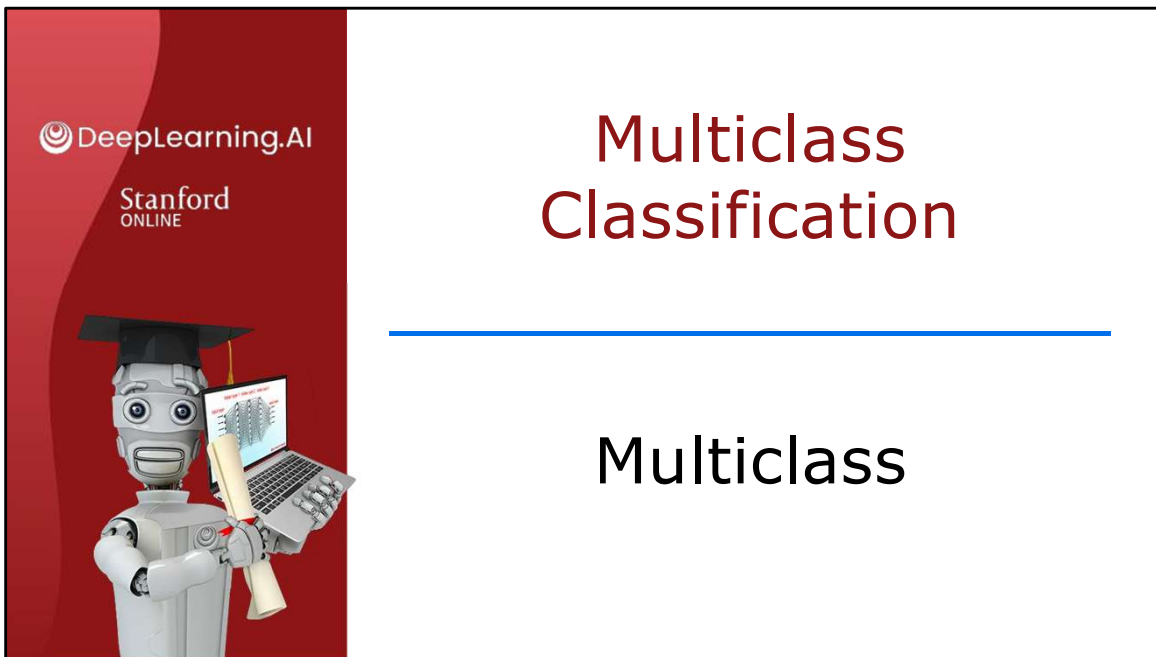
Here, all hidden layers use $g(z) = z$, which means they are linear. When we write the equations step by step, we see that each layer computes a linear function of the previous layer. But a linear function of a linear function is still linear. So even if we stack many layers, the final result is still of the form $f(x) = wx + b$.

This means the whole deep network becomes equivalent to simple linear regression. The extra layers do not add any extra power. The network cannot learn complex patterns because everything stays linear.

On the right side, the output layer uses sigmoid. If only the output is sigmoid and all hidden layers are linear, then the model becomes equivalent to logistic regression. Again, it is not really a deep nonlinear model.

The key idea is that nonlinearity must appear in the hidden layers. That is why we use activation functions like ReLU in hidden layers. ReLU introduces nonlinearity, which allows the network to learn complex relationships and patterns in the data.

Important takeaway: If hidden layers are linear, depth does not help. To make deep learning powerful, we must use nonlinear activation functions in hidden layers.



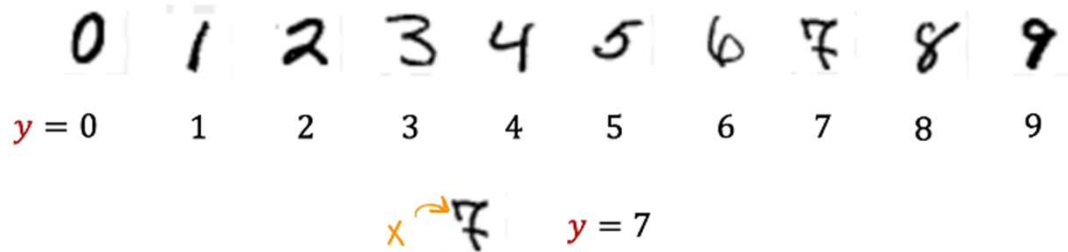
Multiclass Classification

In this section, we move from binary classification, where the output is only 0 or 1, to multiclass classification, where the model must choose one class out of many possible classes. Instead of predicting just yes or no, the neural network now predicts which category an input belongs to, such as digit 0 to 9, or different product types.

In multiclass problems, the output layer usually contains multiple neurons, one for each class. Each neuron represents the score or probability for one class. We will see how to design the output layer and which activation function is appropriate (typically softmax), as well as which loss function to use.

The key idea of this section is to understand how neural networks can handle more than two classes and how the architecture and training process slightly change compared to binary classification.

MNIST example



multiclass classification problem:
target y can take on more than two possible values

DeepLearning.AI Stanford ONLINE

MNIST example

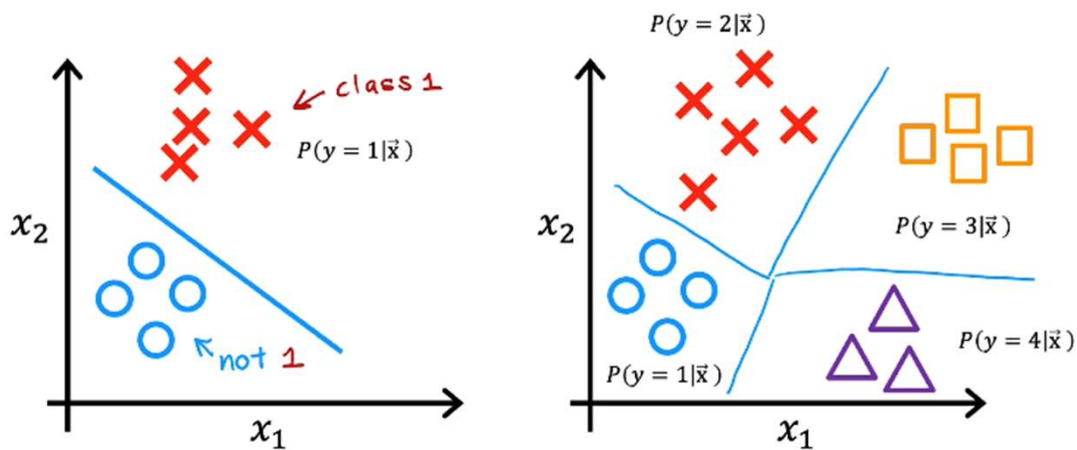
MNIST stands for Modified National Institute of Standards and Technology dataset. It is a large and very popular dataset of handwritten digit images used to train and test machine learning models. It contains 70,000 grayscale images of digits from 0 to 9, where each image is 28 by 28 pixels.

In this example, each input x is one handwritten digit image converted into pixel values. The target y is the correct digit label for that image.

This is a multiclass classification problem because y can take 10 possible values: 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9. For example, if the image looks like the digit 7, then $y = 7$.

The goal of the neural network is to learn from many labeled examples and correctly classify new handwritten digit images.

Multiclass classification example



DeepLearning.AI Stanford ONLINE

Multiclass classification example

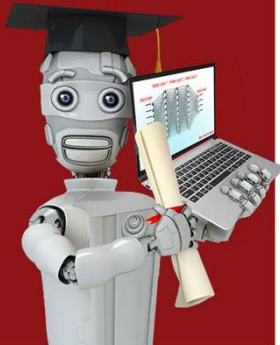
This slide compares binary classification with multiclass classification.

On the left, we have binary classification. The model separates the data into two groups: class 1 and not class 1. The decision boundary is a single line, and the model learns $P(y = 1 | x)$. Everything on one side is predicted as class 1, and everything on the other side is predicted as the other class.

On the right, we have multiclass classification. Now the target y can take more than two values, such as class 1, class 2, class 3, or class 4. Instead of one decision boundary, the model learns multiple regions in the input space. Each region corresponds to a different class.

In multiclass classification, the model estimates probabilities like $P(y = 1 | x)$, $P(y = 2 | x)$, $P(y = 3 | x)$, and $P(y = 4 | x)$. The predicted class is the one with the highest probability.

The key idea is that multiclass classification divides the input space into several regions, not just two, and assigns each region to a different class.



Multiclass Classification

Softmax

Softmax

In multiclass classification, we need an output layer that can handle more than two classes. Softmax is the most common activation function used in the output layer for this purpose.

Softmax takes a vector of numbers (called logits) and converts them into probabilities. Each output value becomes a number between 0 and 1, and all the probabilities add up to 1.

This means the model does not just predict a class label. It gives a probability for each class, such as $P(y = 0 \mid x)$, $P(y = 1 \mid x)$, ..., $P(y = 9 \mid x)$. The predicted class is the one with the highest probability.

The key idea is that Softmax allows the neural network to choose one class out of many by producing a proper probability distribution over all classes.

Logistic regression (2 possible output values)	Softmax regression (4 possible outputs) $y = 1, 2, 3, 4$
$z = \vec{w} \cdot \vec{x} + b$ $\times a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1 \vec{x})$ 0.71 $\circ a_2 = 1 - a_1 = P(y=0 \vec{x})$ 0.29	$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1$ $a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=1 \vec{x})$ $\circ z_2 = \vec{w}_2 \cdot \vec{x} + b_2$ $a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=2 \vec{x})$ $\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3$ $a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=3 \vec{x})$ $\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4$ $a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=4 \vec{x})$

DeepLearning.AI
Stanford ONLINE

This slide compares logistic regression (binary classification) with softmax regression (multiclass classification).

On the left, logistic regression handles only two possible outputs, such as $y = 0$ or $y = 1$. We compute $z = w \cdot x + b$, then apply the sigmoid function $a = 1 / (1 + e^{-(z)})$. The output a gives $P(y = 1 | x)$. Since there are only two classes, $P(y = 0 | x) = 1 - a$. So one output neuron is enough.

On the right, softmax regression is used when there are more than two classes, for example $y = 1, 2, 3, 4$. Instead of one z , we compute one z for each class: $z_1 = w_1 \cdot x + b_1$, $z_2 = w_2 \cdot x + b_2$, $z_3 = w_3 \cdot x + b_3$, $z_4 = w_4 \cdot x + b_4$. Then we apply the softmax formula. Each output $a_i = e^{(z_i)}$ divided by the sum $e^{(z_1)} + e^{(z_2)} + e^{(z_3)} + e^{(z_4)}$.

Each a_i represents $P(y = i | x)$. All probabilities are between 0 and 1, and they add up to 1. The predicted class is the one with the largest probability.

The key idea is that sigmoid is used for binary classification, while softmax is the generalization used for multiclass classification.

Logistic regression (2 possible output values)	Softmax regression (4 possible outputs) $y = 1, 2, 3, 4$
$z = \vec{w} \cdot \vec{x} + b$ $\times a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1 \vec{x})$ 0.71 $\circ a_2 = 1 - a_1 = P(y=0 \vec{x})$ 0.29	$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1$ $a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=1 \vec{x})$ 0.30 $\circ z_2 = \vec{w}_2 \cdot \vec{x} + b_2$ $a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=2 \vec{x})$ 0.20 $\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3$ $a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=3 \vec{x})$ 0.15 $\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4$ $a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=4 \vec{x})$ 0.35

Stanford ONLINE

This slide continues the comparison between logistic regression and softmax regression, now with example probability values.

On the left, logistic regression is used for two possible classes only. We compute $z = w \cdot x + b$, then apply the sigmoid function $a_1 = 1 / (1 + e^{-(z)})$. If $a_1 = 0.71$, this means $P(y = 1 | x) = 0.71$. Since there are only two classes, the other probability is $a_2 = 1 - a_1 = 0.29$, which represents $P(y = 0 | x)$. The two probabilities always add up to 1.

On the right, softmax regression handles four possible classes, $y = 1, 2, 3, 4$. For each class, we compute a separate score: $z_1 = w_1 \cdot x + b_1$, $z_2 = w_2 \cdot x + b_2$, $z_3 = w_3 \cdot x + b_3$, and $z_4 = w_4 \cdot x + b_4$. Then we apply the softmax formula: each $a_i = e^{(z_i)}$ divided by the sum $e^{(z_1)} + e^{(z_2)} + e^{(z_3)} + e^{(z_4)}$.

In the example shown, the output probabilities are:

$$P(y = 1 | x) = 0.30$$

$$P(y = 2 | x) = 0.20$$

$$P(y = 3 | x) = 0.15$$

$$P(y = 4 | x) = 0.35$$

All probabilities are between 0 and 1, and they add up to 1. The predicted class is the one with the highest probability, which here is class 4 (0.35).

The key idea is that sigmoid gives probability for two classes, while softmax gives a probability distribution over multiple classes.

Logistic regression (2 possible output values)	Softmax regression (4 possible outputs) $y=1,2,3,4$
$z = \vec{w} \cdot \vec{x} + b$ $\times a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1 \vec{x})$ 0.71 $\circ a_2 = 1 - a_1 = P(y=0 \vec{x})$ 0.29	$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1$ $a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=1 \vec{x})$ 0.30 $\circ z_2 = \vec{w}_2 \cdot \vec{x} + b_2$ $a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=2 \vec{x})$ 0.20 $\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3$ $a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=3 \vec{x})$ 0.15 $\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4$ $a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=4 \vec{x})$ 0.35
Softmax regression (N possible outputs) $y=1,2,3,\dots,N$ $z_j = \vec{w}_j \cdot \vec{x} + b_j \quad j=1,\dots,N$ <i>parameters</i> $w_1, w_2, \dots, w_N$ $b_1, b_2, \dots, b_N$ $a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y=j \vec{x})$ note: $a_1 + a_2 + \dots + a_N = 1$	

DeepLearning.AI
Stanford ONLINE

This slide compares logistic regression with softmax regression and explains how we move from binary classification to multiclass classification.

On the left side, logistic regression is used when there are only two possible output values, for example $y = 0$ or $y = 1$. First, we compute $z = w \cdot x + b$. Then we apply the sigmoid function: $a = 1 / (1 + e^{(-z)})$. The output a represents $P(y = 1 | x)$. Since there are only two classes, the probability of the other class is $1 - a$, which represents $P(y = 0 | x)$. The two probabilities always add up to 1.

On the right side, softmax regression is used when there are more than two possible classes, for example $y = 1, 2, 3, 4$. Instead of computing one z value, we compute one z_j for each class: $z_j = w_j \cdot x + b_j$. So each class has its own weights and bias.

Then we apply the softmax function. For each class j , the output is $a_j = e^{(z_j)} / (e^{(z_1)} + e^{(z_2)} + \dots + e^{(z_N)})$.

Each a_j represents $P(y = j | x)$. All probabilities are positive and they always sum to 1. This makes softmax a natural extension of logistic regression to multiple classes.

The key idea is that logistic regression handles two classes using sigmoid, while softmax handles N classes by normalizing exponentials of multiple z values into a probability distribution.

Cost

Logistic regression

$$z = \vec{w} \cdot \vec{x} + b$$

$$a_1 = g(z) = \frac{1}{1 + e^{-z}} = P(y = 1 | \vec{x})$$

$$a_2 = 1 - a_1 = P(y = 0 | \vec{x})$$

$$\text{loss} = -y \log a_1 - (1 - y) \log(1 - a_1)$$

if $y = 1$ if $y = 0$

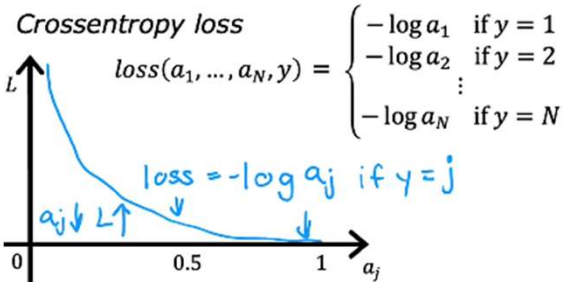
$$J(\vec{w}, b) = \text{average loss}$$

Softmax regression

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = 1 | \vec{x})$$

$$\vdots$$

$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = N | \vec{x})$$



This slide explains the cost function for logistic regression and softmax regression.

On the left side, we review logistic regression for binary classification. We compute $z = w \cdot x + b$, then apply the sigmoid function $a_1 = 1 / (1 + e^{-(z)})$. This gives $P(y = 1 | x)$. Since there are only two classes, $P(y = 0 | x) = 1 - a_1$.

The loss for one example is

$$\text{loss} = -y \log(a_1) - (1 - y) \log(1 - a_1).$$

If $y = 1$, the loss becomes $-\log(a_1)$.

If $y = 0$, the loss becomes $-\log(1 - a_1)$.

So the model is penalized when it assigns low probability to the correct class.

The total cost $J(w, b)$ is the average loss over all training examples.

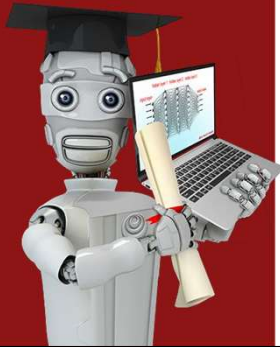
On the right side, we extend this idea to softmax regression for N classes. The softmax outputs $a_1, a_2, \dots, a_N$, where each $a_j = P(y = j | x)$, and all probabilities sum to 1.

For one example, the cross-entropy loss is

$$\text{loss} = -\log(a_j) \text{ if the true class is } j.$$

So we only look at the predicted probability of the correct class. If the model gives a small probability to the correct class, the loss becomes large. If it gives a probability close to 1, the loss becomes small.

The key idea is that both logistic regression and softmax use cross-entropy loss. In binary classification, we use the sigmoid version. In multiclass classification, we use the softmax version.



Multiclass Classification

Neural Network with

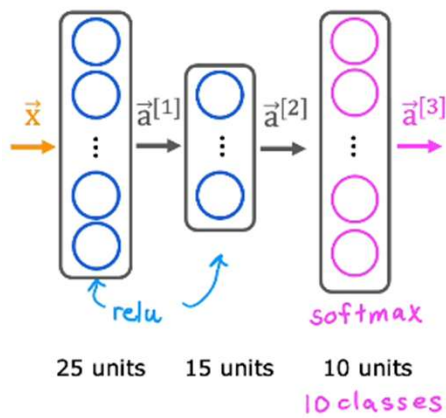
This section introduces how to build and train a neural network for multiclass classification using a softmax output layer.

In binary classification, the output layer has one neuron with a sigmoid activation. But in multiclass classification, the output layer has one neuron per class. Instead of sigmoid, we use the softmax activation function.

Softmax converts the raw outputs (z values) into probabilities for each class. All output probabilities are between 0 and 1, and they always sum to 1. This allows the model to choose the class with the highest probability as the final prediction.

In this section, we will see how the neural network structure changes when using softmax, how the output layer is defined in TensorFlow, and how cross-entropy loss is used together with softmax for training.

Neural Network with Softmax output



DeepLearning.AI Stanford ONLINE

This slide shows the structure of a neural network designed for multiclass classification using a softmax output layer.

The input x passes through two hidden layers. The first hidden layer has 25 units, and the second hidden layer has 15 units. These hidden layers usually use ReLU activation, which helps the network learn complex, non-linear patterns.

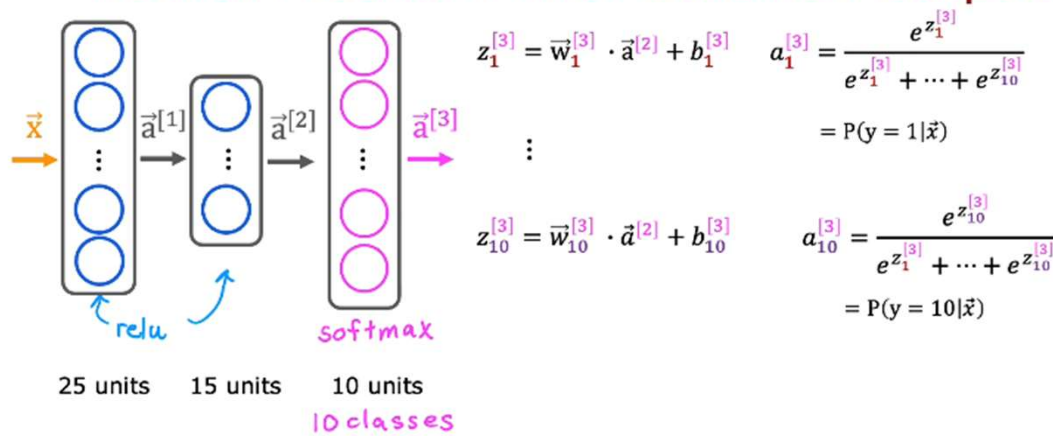
The final layer is the output layer, and it has 10 units. Each unit corresponds to one class. For example, in digit classification, the 10 units represent digits 0 through 9.

Instead of sigmoid, the output layer uses the softmax activation function.

Softmax takes the 10 output values and converts them into 10 probabilities. Each value represents the probability that the input belongs to that class. All probabilities are between 0 and 1, and their sum equals 1.

The predicted class is the one with the highest probability. This architecture allows the network to classify inputs into more than two possible categories.

Neural Network with Softmax output



DeepLearning.AI Stanford ONLINE

This slide explains in more detail how a neural network works when we use a softmax output layer for multiclass classification.

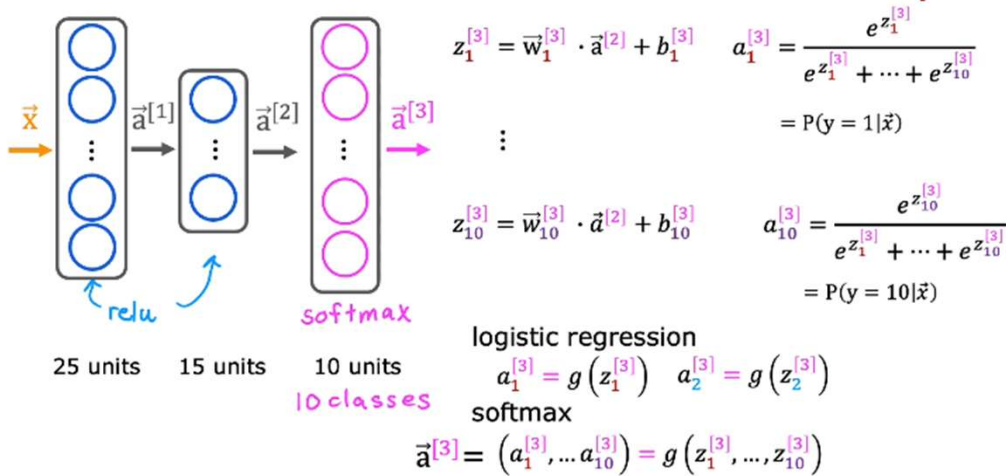
The input x first passes through the hidden layers, which use ReLU activation. The first hidden layer has 25 units, and the second hidden layer has 15 units. These layers learn useful features from the input.

In the output layer, there are 10 neurons, one for each class. For each output neuron j , we first compute a linear value z_j using the formula $z_j = w_j \cdot a^{[2]} + b_j$. This means each class has its own weight vector w_j and bias b_j , and it uses the activations from the previous layer $a^{[2]}$.

After computing all $z_1, z_2, \dots, z_{10}$, we apply the softmax function. Each output activation is computed as $a_j = e^{z_j}$ divided by the sum of e^{z_k} for all classes k from 1 to 10. This converts the raw scores into probabilities. Each a_j represents $P(y = j | x)$, the probability that the input belongs to class j . All probabilities are positive and add up to 1.

Finally, the predicted class is the one with the highest probability. This is why softmax is the standard choice for multiclass classification problems such as digit recognition.

Neural Network with Softmax output



DeepLearning.AI Stanford ONLINE

This slide compares logistic regression and softmax in the output layer of a neural network.

The network structure is the same as before. The input x passes through hidden layers with ReLU activation, producing $a^{[2]}$. Then the output layer has 10 units, one for each class.

For each output neuron j , we compute $z_j^{[3]} = w_j^{[3]} \cdot a^{[2]} + b_j^{[3]}$. These are the raw scores for each class.

In logistic regression (binary classification), each output is computed independently as $a_j = g(z_j)$, where g is the sigmoid function. This works when we have only two classes.

In softmax (multiclass classification), we do not apply sigmoid separately to each z_j . Instead, we apply the softmax function to the entire vector $z^{[3]} = (z_1, \dots, z_{10})$. The output vector $a^{[3]} = (a_1, \dots, a_{10})$ is computed together using $a_j = e^{z_j}$ divided by the sum of e^{z_k} for all classes.

This means softmax looks at all class scores at the same time and converts them into probabilities that sum to 1.

So the key idea is: logistic regression uses sigmoid for binary output, while softmax is the generalization for multiple classes, producing a full probability distribution over all classes.

MNIST with softmax

① specify the model

$$f_{\vec{w},b}(\vec{x}) = ?$$

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=10, activation='softmax')
])
from tensorflow.keras.losses import
    SparseCategoricalCrossentropy
model.compile(loss= SparseCategoricalCrossentropy() )
model.fit(X,Y,epochs=100)
```

② specify loss and cost

$$L(f_{\vec{w},b}(\vec{x}), y)$$

③ Train on data to minimize $J(\vec{w},b)$

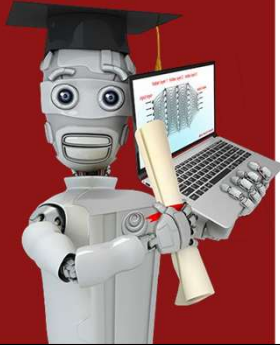
Note: better (recommended) version later.
Don't use the version shown here!

This slide shows how to train a neural network for MNIST using a softmax output layer.

First, we define the model. We import TensorFlow and use Sequential to stack layers. The model has two hidden layers with ReLU activation, for example `Dense(units=25, activation='relu')` and `Dense(units=15, activation='relu')`. The output layer is `Dense(units=10, activation='softmax')`. The 10 units correspond to the 10 digit classes from 0 to 9. The softmax activation makes the output a probability distribution over the 10 classes.

Second, we specify the loss function. Since MNIST is a multiclass classification problem and the labels are integers (0 to 9), we use `SparseCategoricalCrossentropy`. This loss compares the predicted probability for the correct class with the true label and penalizes the model if it assigns low probability to the correct digit. We set this using `model.compile(loss=SparseCategoricalCrossentropy())`.

Third, we train the model using `model.fit(X, Y, epochs=100)`. During training, gradient descent and backpropagation adjust the weights to reduce the loss. The key idea is that for multiclass problems like MNIST, we use a softmax output layer with 10 units and a categorical cross-entropy type loss to train the network.



Multiclass Classification

Improved implementation of softmax

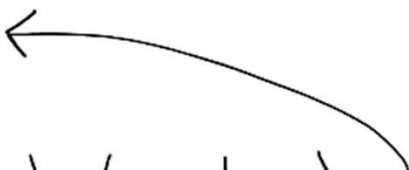
This section introduces **multiclass classification**, where a model must choose one class from more than two possible classes (for example, classifying digits 0–9 instead of just yes/no). To solve this problem, we use the **softmax function**, which converts model outputs into probabilities that sum to 1. In this section, we will focus on an improved and more stable implementation of softmax, which is commonly used in neural networks for efficient and accurate multiclass prediction.

Numerical Roundoff Errors

option 1

$$x = \frac{2}{10,000}$$

option 2

$$x = \left(1 + \frac{1}{10,000}\right) - \left(1 - \frac{1}{10,000}\right) =$$


This slide explains the idea of **numerical roundoff errors**, which happen because computers cannot store numbers with perfect precision.

It shows two different ways to compute the same value:

Option 1:

$$x = 2 / 10,000$$

Option 2:

$$x = (1 + 1 / 10,000) - (1 - 1 / 10,000)$$

Mathematically, both expressions give exactly the same result. However, inside a computer, the second method can produce a small numerical error. This happens because we subtract two numbers that are very close to each other. When very similar numbers are subtracted, the computer may lose important digits. This is called **loss of precision** or **roundoff error**.

This concept is very important in machine learning. Functions like softmax use exponentials, which can create very large or very small numbers. If we are not careful, numerical roundoff errors can make the computation unstable. That is why we need numerically stable implementations of algorithms like softmax.



```
In [1]: x1 = 2.0 / 10000
print(f"{x1:.18f}") # print 18 digits to the right of decimal point

0.00020000000000000000
```

```
In [2]: x2 = 1 + (1/10000) - (1 - 1/10000)
print(f"{x2:.18f}")

0.00019999999999999978
```

```
In [ ]:
```

Numerical Roundoff Errors

More numerically accurate implementation of logistic loss:

Logistic regression:

$$\hat{a} = g(z) = \frac{1}{1 + e^{-z}}$$

Original loss

$$\text{loss} = -y \log(\hat{a}) - (1 - y) \log(1 - \hat{a})$$

More accurate loss (in code)

$$\text{loss} = -y \log\left(\frac{1}{1 + e^{-z}}\right) - (1 - y) \log\left(1 - \frac{1}{1 + e^{-z}}\right)$$

```
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'), 'linear'
    Dense(units=1, activation='sigmoid')
])
model.compile(loss=BinaryCrossEntropy())
model.compile(loss=BinaryCrossEntropy(from_logits=True))
```

This slide shows a practical example in Jupyter Notebook to demonstrate **numerical roundoff error**.

In the first cell, the value is calculated directly as:

$$x1 = 2.0 / 10000$$

The result printed with 18 decimal digits is:

0.00020000000000000000

This looks exact and correct.

In the second cell, the same value is computed using:

$$x2 = (1 / 10000) - (1 - 1 / 10000)$$

Mathematically, this should give exactly the same result as x1. However, when printed with 18 decimal digits, the result becomes:

0.00019999999999999978

We can see a very small difference in the last digits.

This difference appears because the computer stores numbers with limited precision. When subtracting two numbers that are very close to each other (like $1 + \text{small value}$ and $1 - \text{small value}$), some important digits are lost. This is called **loss of precision** or **roundoff error**.

This example clearly shows why numerical stability is important in machine learning. When algorithms (like softmax) involve exponentials and subtraction of large or very similar numbers, small numerical errors can appear. That is why we use improved and numerically stable implementations.

More numerically accurate implementation of softmax

Softmax regression

$$(a_1, \dots, a_{10}) = g(z_1, \dots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=10, activation='softmax')
])
```

'linear'

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

```
model.compile(loss=SparseCategoricalCrossEntropy())
```

```
model.compile(loss=SparseCategoricalCrossEntropy(from_logits=True))
```

DeepLearning.AI Stanford ONLINE

This slide explains how to implement **softmax in a more numerically accurate way**.

In softmax regression, the model outputs several values (for example, 10 values for digits 0–9). These values are converted into probabilities using the softmax function. The loss function then calculates the negative log of the predicted probability of the correct class.

However, computing softmax directly can cause numerical problems. This is because softmax uses exponential functions like e raised to z . If the values of z are very large or very small, the exponential can overflow (become extremely large) or underflow (become extremely small). This leads to instability and inaccurate loss calculations.

To make the implementation more accurate, we avoid computing softmax separately before the loss. Instead, we use the logits (the raw outputs before softmax) directly inside a special loss function. In TensorFlow, this is done by setting the final layer activation to linear (no softmax) and using:

```
loss = SparseCategoricalCrossEntropy(from_logits=True)
```

This allows the framework to combine softmax and cross-entropy in a single, numerically stable computation.

The key idea of this slide is:

For better numerical stability, do not apply softmax manually. Instead, pass the raw logits to a stable cross-entropy implementation that handles softmax internally in a safer way.

MNIST (more numerically accurate)

```
model    import tensorflow as tf
         from tensorflow.keras import Sequential
         from tensorflow.keras.layers import Dense
         model = Sequential([
             Dense(units=25, activation='relu'),
             Dense(units=15, activation='relu'),
             Dense(units=10, activation='linear') ])

loss     from tensorflow.keras.losses import
         SparseCategoricalCrossentropy

         model.compile(..., loss=SparseCategoricalCrossentropy(from_logits=True) )

fit      model.fit(X,Y,epochs=100)

predict  logits = model(X) ← not  $a_1 \dots a_{10}$ 
         f_x = tf.nn.softmax(logits)      is  $z_1 \dots z_{10}$ 
```

DeepLearning.AI Stanford ONLINE

This slide shows how to implement the MNIST digit classification model in a **more numerically accurate way**.

The model is built using TensorFlow with two hidden layers using ReLU activation. The final layer has 10 units (for digits 0–9), but notice that the activation is set to **linear**, not softmax.

This is very important.

Instead of applying softmax inside the model, we compile the model using: `SparseCategoricalCrossEntropy(from_logits=True)`

This means that the loss function expects **logits** (raw outputs z_1 to z_{10}), not probabilities (a_1 to a_{10}). The framework will internally apply softmax in a numerically stable way when computing the loss.

During training:

- The model outputs logits.
- The loss function handles softmax safely.

During prediction:

- `model(X)` returns logits (z_1 to z_{10}).
- If we want probabilities, we manually apply softmax using:
`tf.nn.softmax(logits)`

The key idea of this slide is:

For better numerical stability, do not use softmax as the final activation when training. Instead, use a linear output layer and let the cross-entropy loss handle softmax internally with `from_logits=True`. This avoids numerical roundoff problems and makes training more stable and accurate.

logistic regression (more numerically accurate)

```
model    model = Sequential([
          Dense(units=25, activation='sigmoid'),
          Dense(units=15, activation='sigmoid'),
          Dense(units=1, activation='linear')
        ])
          from tensorflow.keras.losses import
            BinaryCrossentropy
loss      model.compile(..., BinaryCrossentropy(from_logits=True))
          model.fit(X,Y,epochs=100)
fit        logit = model(X)
predict    f_x = tf.nn.sigmoid(logit)
```

A blue arrow points from the variable 'logit' in the 'fit' line to the 'logit' argument in the 'tf.nn.sigmoid(logit)' function in the 'predict' line.

DeepLearning.AI Stanford ONLINE

This slide shows how to implement **logistic regression in a more numerically accurate way**.

The model has hidden layers using sigmoid activation, but the final output layer uses a **linear activation**, not sigmoid. This means the model outputs a raw value called a **logit** (z), not a probability.

When compiling the model, we use:

`BinaryCrossentropy(from_logits=True)`

This tells TensorFlow that the model outputs logits, and the loss function will internally apply the sigmoid function in a numerically stable way when computing the loss.

During training:

- The model outputs a logit (z).
- The loss function safely applies sigmoid inside the cross-entropy calculation.

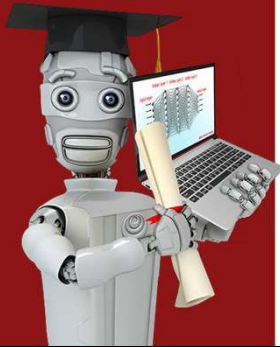
During prediction:

- `model(X)` returns the logit (z).
- If we want a probability between 0 and 1, we manually apply:

`tf.nn.sigmoid(logit)`

The key idea is the same as in softmax:

For better numerical stability, do not apply sigmoid inside the model during training. Instead, use a linear output layer and set `from_logits=True` in the loss function. This avoids numerical roundoff problems and makes training more stable and accurate.



Multi-label Classification

Classification with multiple outputs (Optional)

This section introduces **multi-label classification**, which is different from multiclass classification. In multiclass classification, each example belongs to only one class. In multi-label classification, each example can belong to **multiple classes at the same time**. For example, an image may contain both a car and a person, or a medical case may have more than one condition. In this section, we learn how to design models that produce multiple independent outputs and how to handle classification problems where more than one label can be true simultaneously.

Multi-label Classification



Is there a car? yes no yes
Is there a bus? no no yes
Is there a pedestrian yes yes no

$$y = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \quad y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad y = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

DeepLearning.AI Stanford ONLINE

This slide explains the idea of **multi-label classification** using images as examples.

In each image, we do not ask only one question like “What object is this?” Instead, we ask several independent questions:

- Is there a car?
- Is there a bus?
- Is there a pedestrian?

Each question has its own yes (1) or no (0) answer. So instead of producing one single class label, the model produces a **vector of outputs**, for example:

- [1, 0, 1] → car: yes, bus: no, pedestrian: yes
- [0, 0, 1] → only pedestrian
- [1, 1, 0] → car and bus, no pedestrian

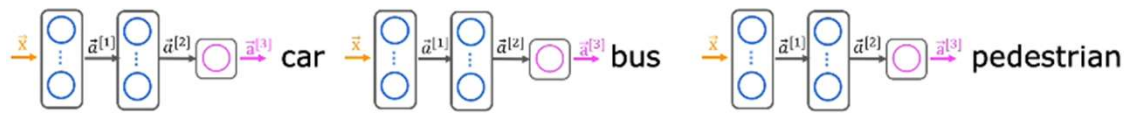
The important idea is that the labels are **not mutually exclusive**. An image can contain more than one object at the same time.

In multi-label classification:

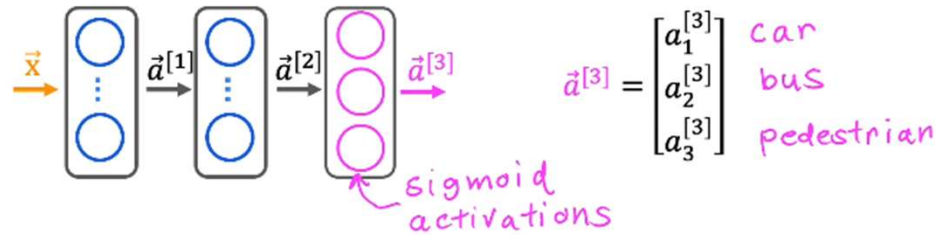
- Each output unit makes an independent decision.
- We usually use one sigmoid output per label.
- The model predicts multiple binary values instead of one single class.

This is different from multiclass classification, where only one class can be correct.

Multi-label Classification



Alternatively, train one neural network with three outputs



This slide explains two ways to build a model for **multi-label classification**. In the first approach, we can train **three separate neural networks**, one for each task:

- One network predicts if there is a car.
- One network predicts if there is a bus.
- One network predicts if there is a pedestrian.

Each network produces one binary output (yes or no).

Alternatively, we can train **one single neural network with three outputs**. In this case:

- The input passes through shared hidden layers.
- The final layer has three output units.
- Each output uses a sigmoid activation.
- Each output independently predicts one label.

The final output is a vector like:

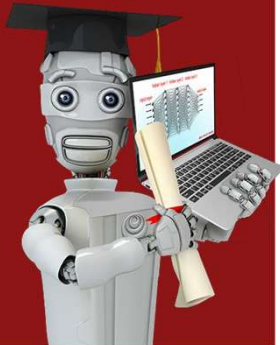
$[a_1, a_2, a_3]$

Where:

- $a_1 \rightarrow$ probability of car
- $a_2 \rightarrow$ probability of bus
- $a_3 \rightarrow$ probability of pedestrian

The key idea is that in multi-label classification:

- Outputs are independent.
- We use sigmoid activation for each output.
- We do not use softmax because multiple labels can be true at the same time.

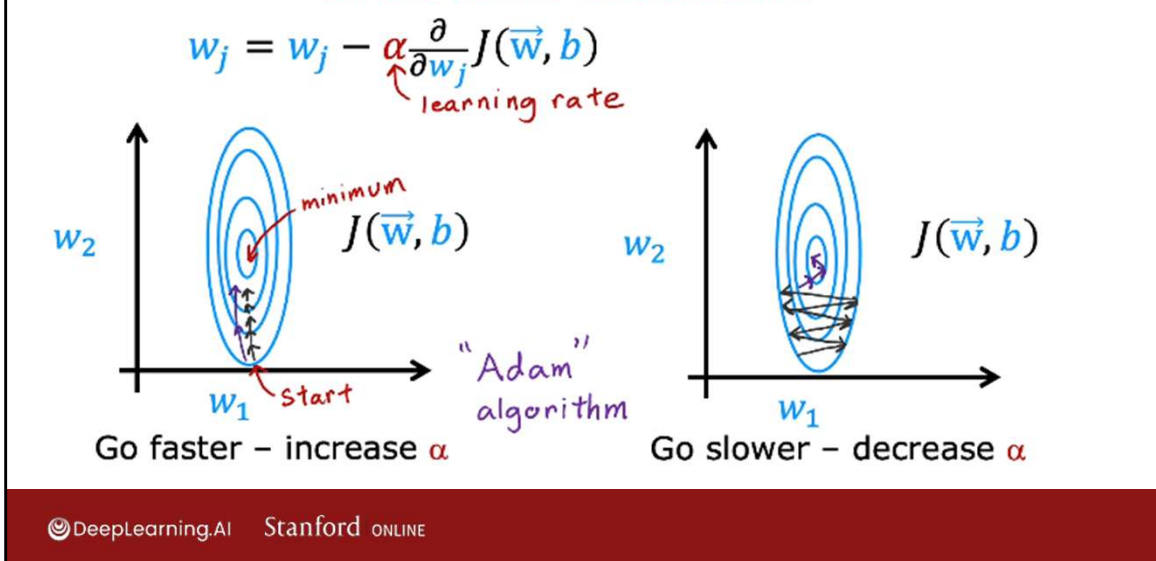


Additional Neural Network Concepts

Advanced Optimization

This section introduces **additional neural network concepts**, focusing on **advanced optimization techniques**. So far, we have used basic gradient descent to train neural networks. In this part, we will explore improved optimization methods that help models train faster, converge more reliably, and achieve better performance. These techniques are especially important for deep neural networks, where training can be slow or unstable without advanced optimization strategies.

Gradient Descent



This slide reviews **Gradient Descent** and highlights the role of the **learning rate (alpha)** in optimization.

The update rule shows that each parameter w_j is updated by subtracting alpha multiplied by the partial derivative of the cost function J with respect to that parameter. The learning rate alpha controls the size of each step taken toward the minimum.

The contour plots illustrate how gradient descent moves from a starting point toward the minimum of the cost function.

- If alpha is increased, the steps become larger, and the algorithm moves faster. However, if alpha is too large, it may overshoot the minimum or become unstable.
- If alpha is decreased, the steps become smaller, and the algorithm moves more slowly, which may make convergence very slow.

The slide also hints at advanced optimization methods (such as Adam), which adjust the step sizes automatically to improve convergence speed and stability compared to basic gradient descent.

The key idea is that choosing the right learning rate is crucial for efficient and stable training.

Adam Algorithm Intuition

Adam: Adaptive Moment estimation *not just one α*

$$\begin{aligned}w_1 &= w_1 - \underbrace{\alpha_1}_{\text{adaptive}} \frac{\partial}{\partial w_1} J(\vec{w}, b) \\&\vdots \\w_{10} &= w_{10} - \underbrace{\alpha_{10}}_{\text{adaptive}} \frac{\partial}{\partial w_{10}} J(\vec{w}, b) \\b &= b - \underbrace{\alpha_{11}}_{\text{adaptive}} \frac{\partial}{\partial b} J(\vec{w}, b)\end{aligned}$$

DeepLearning.AI Stanford ONLINE

This slide introduces the intuition behind the **Adam optimization algorithm**. Adam stands for **Adaptive Moment Estimation**. Unlike standard gradient descent, Adam does not use just one single learning rate for all parameters. In basic gradient descent, the update rule is:

$$w_j = w_j - \alpha * (\partial J(w, b) / \partial w_j)$$

$$b = b - \alpha * (\partial J(w, b) / \partial b)$$

Here, the same learning rate α is used for every parameter.

In Adam, the idea is different. Each parameter has its own adaptive learning rate. Conceptually, the updates look like:

$$w_1 = w_1 - \alpha_1 * (\partial J(w, b) / \partial w_1)$$

$$w_{10} = w_{10} - \alpha_{10} * (\partial J(w, b) / \partial w_{10})$$

$$b = b - \alpha_{11} * (\partial J(w, b) / \partial b)$$

So instead of one α , we effectively have different step sizes for different parameters.

More formally, Adam keeps track of two quantities:

1. The first moment (moving average of gradients):

$$m_t = \beta_1 * m_{(t-1)} + (1 - \beta_1) * g_t$$

2. The second moment (moving average of squared gradients):

$$v_t = \beta_2 * v_{(t-1)} + (1 - \beta_2) * (g_t)^2$$

Where g_t is the gradient at step t .

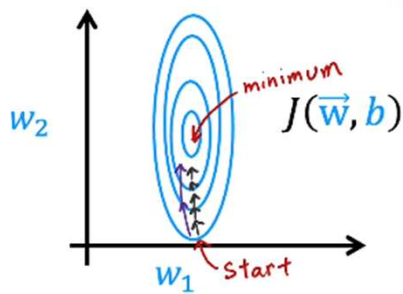
After bias correction, the parameter update becomes:

$$w = w - \alpha * (m_{\text{hat}} / (\sqrt{v_{\text{hat}}} + \epsilon))$$

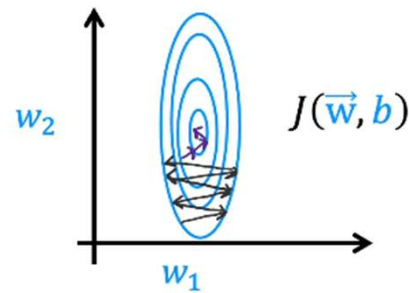
Because of this adaptive scaling using m_{hat} and v_{hat} , Adam adjusts the learning rate for each parameter automatically. This helps training converge faster and more stably, especially in deep neural networks with many

parameters.

Adam Algorithm Intuition



If w_j (or b) keeps moving in same direction, increase α_j .



If w_j (or b) keeps oscillating, reduce α_j .

This slide gives more intuition about how the **Adam algorithm** adapts the learning rate for each parameter.

On the left side, we see a case where a parameter (such as w_j or b) keeps moving in the **same direction** toward the minimum. This means the gradient is consistent. In this situation, Adam will **increase the effective learning rate (α_j)** for that parameter. As a result, the algorithm moves faster in that direction and reaches the minimum more quickly.

On the right side, we see a case where a parameter keeps **oscillating** back and forth. This means the updates are unstable and the gradient direction keeps changing. In this case, Adam will **reduce the effective learning rate (α_j)** for that parameter. This helps stabilize the updates and reduces oscillation.

The key idea is that Adam automatically adjusts the step size for each parameter:

- If movement is stable and consistent $\rightarrow$ increase step size.
- If movement is unstable or oscillating $\rightarrow$ decrease step size.

This adaptive behavior makes Adam faster and more stable than standard gradient descent, especially in deep neural networks.

MNIST Adam

model

```
model = Sequential([  
    tf.keras.layers.Dense(units=25, activation='sigmoid'),  
    tf.keras.layers.Dense(units=15, activation='sigmoid'),  
    tf.keras.layers.Dense(units=10, activation='linear')  
])
```

compile

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),  
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

fit

```
model.fit(X, Y, epochs=100)
```

$$\alpha = 10^{-3} = 0.001$$

This slide shows how to train the **MNIST digit classification model using the Adam optimizer**.

The model structure is similar to before:

- Two hidden layers with sigmoid activation.
- A final layer with 10 units (for digits 0–9) and **linear activation**.
- We use `SparseCategoricalCrossEntropy(from_logits=True)` because the output layer produces logits.

The important difference in this slide is the optimizer:

```
optimizer = tf.keras.optimizers.Adam(learning_rate = 1e-3)
```

Here, the learning rate alpha is set to:

$$\alpha = 10^{-3} = 0.001$$

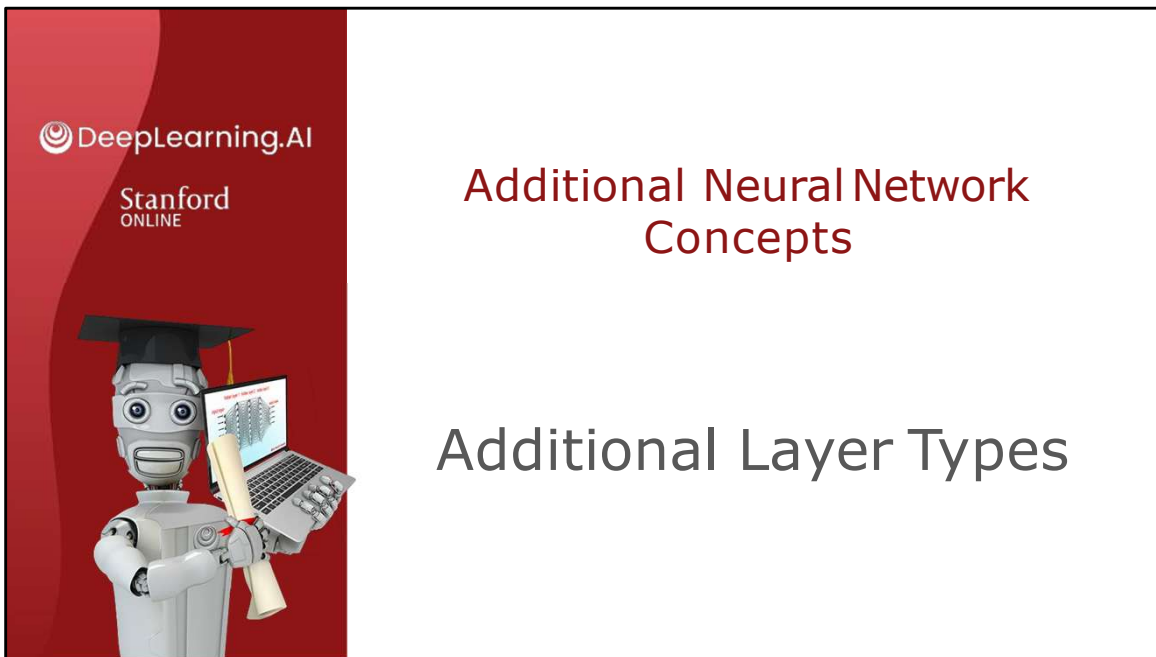
Instead of using basic gradient descent, we now use **Adam**, which:

- Automatically adapts the learning rate for each parameter.
- Converges faster.
- Is more stable during training.

Finally, we train the model using:

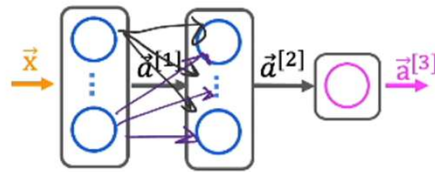
```
model.fit(X, Y, epochs=100)
```

The key idea of this slide is that Adam is a practical and powerful optimizer. In real-world deep learning applications, Adam is often preferred over standard gradient descent because it trains faster and requires less manual tuning of the learning rate.



This section introduces **additional layer types in neural networks**. So far, we have mainly worked with fully connected (Dense) layers. In this part, we will explore other types of layers that are commonly used in deep learning models. Different layer types are designed for different tasks, such as image processing, sequence data, or improving model performance. Understanding these additional layers helps us build more powerful and specialized neural network architectures.

Dense Layer



Each neuron output is a function of all the activation outputs of the previous layer.

$$\vec{a}_1^{[2]} = g(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]})$$

This slide explains the **Dense layer**, also called a **fully connected layer**.

In a dense layer, each neuron is connected to **all neurons in the previous layer**. This means that every output in the current layer depends on all activation values from the previous layer.

Mathematically, the output of one neuron in layer 2 can be written as:

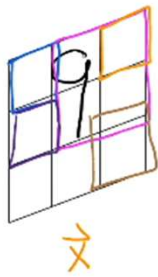
$$a_1^{(2)} = g(w_1^{(2)} \cdot a^{(1)} + b_1^{(2)})$$

This means:

- Take all activations from the previous layer ($a^{(1)}$).
- Multiply them by their corresponding weights.
- Add a bias term.
- Apply an activation function g (such as ReLU, sigmoid, etc.).

Because every neuron is connected to every neuron in the previous layer, this layer can learn complex combinations of features. Dense layers are commonly used in many neural networks, especially for general-purpose tasks like classification and regression.

Convolutional Layer



Each neuron only looks at part of the previous layer's outputs.

Why?

- Faster computation
- Need less training data (less prone to overfitting)

This slide introduces the **Convolutional Layer**, which is commonly used in image processing tasks.

Unlike a dense layer, where each neuron looks at all outputs from the previous layer, in a convolutional layer each neuron looks at **only a small part** of the input. This small part is called a **local region** or **receptive field**.

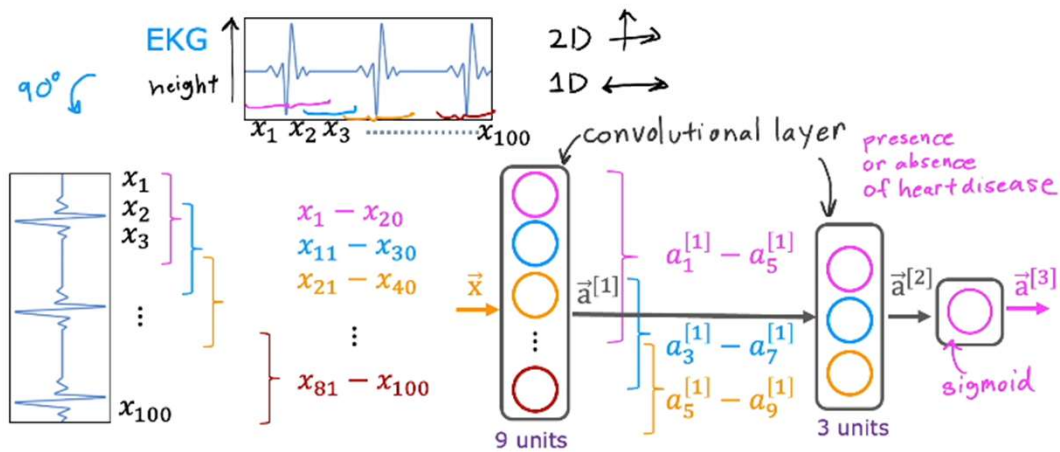
For example, when analyzing an image, a neuron may look at just a small patch of pixels instead of the entire image. As the filter moves across the image, it detects patterns such as edges, shapes, or textures.

Why is this useful?

- **Faster computation:** Fewer connections mean fewer parameters.
- **Needs less training data:** Because the model has fewer parameters.
- **Less prone to overfitting:** The model is simpler and focuses on local patterns.

The key idea is that convolutional layers are designed to efficiently detect meaningful patterns in structured data like images, making them very powerful for computer vision tasks.

Convolutional Neural Network



DeepLearning.AI Stanford ONLINE

This slide explains the structure of a **Convolutional Neural Network (CNN)** using a simple 1D example.

On the left, we see an input signal (for example, ECG data). Instead of connecting every input value to every neuron (like in a dense layer), the convolutional layer looks at **small segments of the input** at a time (for example, $x_1 - x_{20}$, $x_{11} - x_{30}$, $x_{21} - x_{40}$, etc.). These small segments overlap and move across the input. This process is called **convolution**.

Each filter produces a set of activations (for example, 9 units). These activations detect specific patterns in the signal, such as shapes or important features.

After the convolutional layer, the outputs are passed to another layer (for example, 3 units), and finally to an output neuron with a sigmoid activation. In this example, the final output may represent something like the **presence or absence of heart disease**.

The key idea is:

- Convolutional layers detect local patterns.
- These patterns are combined in later layers.
- The network learns meaningful features automatically from raw input data.

CNNs are widely used for images, signals, audio, and other structured data because they are efficient and powerful at detecting patterns.

Modern Neural Network Architectures and What's Next

- Modern architectures like **Transformers, LSTMs, and Attention models** are based on designing new types of neural network layers.
- Current research in deep learning focuses on **inventing new layer types**.
- These layers are used as **building blocks** to create more powerful and complex neural networks.
- Neural network design is an evolving field, with continuous improvements in architecture.

This slide explains that modern neural network progress comes mainly from designing **new types of layers**. Advanced architectures such as Transformers, LSTMs, and Attention models are not completely different from earlier neural networks. Instead, they are built by introducing new layer designs and combining them in more effective ways. These new layers are created to handle specific challenges, such as processing sequences, focusing on important information, or capturing long-range dependencies in data. Much of today's research in deep learning focuses on inventing new layer types. Researchers experiment with different mathematical structures and connection patterns to improve performance, stability, and efficiency. Once a new layer is developed, it can be used as a building block inside larger neural network architectures.

Neural network design is therefore an evolving field. Instead of only increasing data or model size, researchers improve how networks are structured. By combining different layers creatively, they build more powerful and complex systems. This continuous innovation in architecture design is one of the main reasons why deep learning keeps advancing so rapidly.